Kotlin & Jetpack Compose Developer Guide

Volume 6

SharedPreferences | Gson | JSON Serialization

Storing Complex Objects | Order History Pattern

Flutter vs Kotlin Comparison

For Flutter/Dart developers learning Kotlin & Jetpack Compose

Chapter 1: What is SharedPreferences?

The Core Concept

SharedPreferences is Android's built-in key-value storage system. It persists small amounts of data across app restarts -- like a tiny local database that survives when the app is closed. No setup, no migrations, no SQL.

Think of it exactly like a Map<String, Any> that is automatically saved to a file on the device and reloaded every time the app opens.

Flutter Equivalent

In Flutter/Dart the direct equivalent is the 'shared_preferences' package. Both work identically -- key-value pairs persisted to disk.

Concept	Flutter / Dart	Android / Kotlin
Package/API	shared_preferences package	Built-in (no dependency)
Get instance	SharedPreferences.getInstance()	context.getSharedPreferences(name,
Write String	prefs.setString('k', 'v')	prefs.edit().putString('k', 'v').apply()
Read String	prefs.getString('k') ?? ''	prefs.getString('k', '')
Delete key	prefs.remove('k')	prefs.edit().remove('k').apply()
Clear all	prefs.clear()	prefs.edit().clear().apply()

What Types Can Be Stored

[!] SharedPreferences can ONLY store primitive types. Complex objects like Order, Product, or List<Order> cannot be stored directly -- they must be converted to a String first (JSON).

Type	Kotlin method	Dart method
String	putString(key, value)	setString(key, value)
Int	putInt(key, value)	setInt(key, value)
Boolean	putBoolean(key, value)	setBool(key, value)
Float/Double	putFloat(key, value)	setDouble(key, value)
Long	putLong(key, value)	N/A (use int)
Complex object	NOT supported -- use JSON String	NOT supported -- use JSON String

apply() vs commit()

Every write to SharedPreferences goes through an Editor. You end the edit with either apply() or commit():

Kotlin / Jetpack Compose

```
prefs.edit().putString("key", "value").apply()   // async -- fire and forget, no result
prefs.edit().putString("key", "value").commit()  // sync  -- blocks thread, returns Boolean
```

[NOTE] Always use apply() unless you specifically need to know if the write succeeded. commit() blocks the calling thread which can freeze the UI if called on the main thread.

SharedPreferencesManager in This Project

Rather than calling SharedPreferences directly everywhere, the project wraps it in a @Singleton class called SharedPreferencesManager. This follows the same pattern as wrapping shared_preferences in a service class in Flutter.

Kotlin / Jetpack Compose

```
@Singleton
class SharedPreferencesManager @Inject constructor(
    @ApplicationContext private val context: Context,
) {
    private val prefs by lazy {
        context.getSharedPreferences("app_prefs", Context.MODE_PRIVATE)
    }

    fun putString(key: String, value: String) = prefs.edit().putString(key, value).apply()
    fun getString(key: String, default: String = "") = prefs.getString(key, default) ?: default
    fun putInt(key: String, value: Int) = prefs.edit().putInt(key, value).apply()
    fun getInt(key: String, default: Int = 0) = prefs.getInt(key, default)
    fun putBoolean(key: String, value: Boolean) = prefs.edit().putBoolean(key, value).apply()
    fun getBoolean(key: String, default: Boolean = false) = prefs.getBoolean(key, default)
    fun remove(key: String) = prefs.edit().remove(key).apply()
    fun clear() = prefs.edit().clear().apply()
    fun contains(key: String) = prefs.contains(key)
}
```

Dart / Flutter

```
// Flutter equivalent -- a singleton service wrapping shared_preferences
class LocalStorageService {
  static LocalStorageService? _instance;
  late SharedPreferences _prefs;

  static Future<LocalStorageService> getInstance() async {
    _instance ??= LocalStorageService();
    _instance!._prefs = await SharedPreferences.getInstance();
    return _instance!;
  }

  void putString(String key, String value) => _prefs.setString(key, value);
  String getString(String key, {String defaultValue = ''}) =>
      _prefs.getString(key) ?? defaultValue;
  void remove(String key) => _prefs.remove(key);
  void clear() => _prefs.clear();
}
```

Chapter 2: What is Gson?

The Problem Gson Solves

SharedPreferences cannot store complex objects. But you often need to persist things like Order, Product, List<CartItem>, etc. The solution: convert the object to a JSON string (serialize), store that string, and convert it back when reading (deserialize).

```
Order object  -->  JSON string  -->  stored in SharedPreferences
SharedPreferences  -->  JSON string  -->  Order object
```

Gson is Google's library that handles this conversion automatically. You don't write any parsing code -- Gson inspects the class fields using reflection and builds the JSON for you.

Flutter Equivalent

In Flutter/Dart the equivalent is dart:convert with jsonEncode/jsonDecode plus a fromJson/toJson method on each class. Gson automates what Dart requires you to write manually.

Operation	Dart / Flutter	Kotlin / Gson
Object to JSON	jsonEncode(obj.toJson())	gson.toJson(obj)
JSON to object	MyClass.fromJson(jsonDecode(json))	gson.fromJson(json,
List to JSON	jsonEncode(list.map((e) => e.toJson()).toList())	gson.toJson(list)
JSON to List	manual loop with fromJson	gson.fromJson(json, TypeToken)
Manual toJson needed?	Yes -- you write it	No -- Gson auto-generates

Basic Gson Usage

Kotlin / Jetpack Compose

```
import com.google.gson.Gson

data class Product(val id: Int, val name: String, val price: Double)

val gson = Gson()
val product = Product(id = 1, name = "Phone", price = 99.0)

// Object -> JSON string
val json: String = gson.toJson(product)
// Result: {"id":1,"name":"Phone","price":99.0}

// JSON string -> Object
val restored: Product = gson.fromJson(json, Product::class.java)
println(restored.name)  // Phone
```

Dart / Flutter

```
import 'dart:convert';
```

```
class Product {
  final int id;
  final String name;
  final double price;
  Product({required this.id, required this.name, required this.price});

  // Dart requires you to write these manually -- Gson does this automatically
  Map<String, dynamic> toJson() => {'id': id, 'name': name, 'price': price};
  factory Product.fromJson(Map<String, dynamic> j) =>
      Product(id: j['id'], name: j['name'], price: j['price']);
}

final product = Product(id: 1, name: 'Phone', price: 99.0);

// Object -> JSON string
final json = jsonEncode(product.toJson());
// Result: {"id":1,"name":"Phone","price":99.0}

// JSON string -> Object
final restored = Product.fromJson(jsonDecode(json));
```

Gson with Nested Objects

Gson handles nested objects automatically. If Order contains List<CartItem> and CartItem contains Product, Gson serializes and deserializes the full tree without any extra code.

Kotlin / Jetpack Compose

```
data class CartItem(val product: Product, val quantity: Int)
data class Order(val id: Int, val items: List<CartItem>, val totalPrice: Double)

val order = Order(
    id = 1,
    items = listOf(CartItem(product = Product(1, "Phone", 99.0), quantity = 2)),
    totalPrice = 198.0
)

// Gson handles the nested List<CartItem> automatically
val json = gson.toJson(order)
// Result:
// {
//   "id": 1,
//   "totalPrice": 198.0,
//   "items": [
//     { "quantity": 2, "product": { "id": 1, "name": "Phone", "price": 99.0 } }
//   ]
// }
```

```
val restored: Order = gson.fromJson(json, Order::class.java)
```

Dart / Flutter

```
// Dart -- you must write fromJson/toJson for EVERY class in the tree
class CartItem {
  Map<String, dynamic> toJson() => {'product': product.toJson(), 'quantity': quantity};
  factory CartItem.fromJson(Map<String, dynamic> j) => CartItem(
    product: Product.fromJson(j['product']),
    quantity: j['quantity'],
  );
}

class Order {
  Map<String, dynamic> toJson() => {
    'id': id,
    'items': items.map((e) => e.toJson()).toList(),
    'totalPrice': totalPrice,
  };
  factory Order.fromJson(Map<String, dynamic> j) => Order(
    id: j['id'],
    items: (j['items'] as List).map((e) => CartItem.fromJson(e)).toList(),
    totalPrice: j['totalPrice'],
  );
}
```

[NOTE] This is one of the biggest productivity differences: Gson reflects on your data class fields automatically. Dart requires you to write toJson/fromJson for every class by hand, or use code generation (json_serializable + build_runner).

Chapter 3: TypeToken -- Why Lists Need Special Treatment

The Problem: Type Erasure

Deserializing a single object is simple: gson.fromJson(json, Order::class.java). But deserializing a List<Order> has a problem -- at runtime, Java/Kotlin erases the generic type. The JVM cannot tell the difference between List<Order>, List<String>, or List<CartItem> at runtime -- they all look like just 'List'.

```
// This does NOT work correctly for generic types
val orders: List<Order> = gson.fromJson(json, List::class.java)
// Gson produces a List<LinkedTreeMap> instead of List<Order>
// Because at runtime List::class.java has no info about the <Order> part
```

The Solution: TypeToken

TypeToken captures the full generic type at compile time and passes it to Gson. It is Gson's workaround for Java/Kotlin type erasure.

Kotlin / Jetpack Compose

```
import com.google.gson.reflect.TypeToken

// TypeToken preserves the generic type <List<Order>> through erasure
val type = object : TypeToken<List<Order>>() {}.type

// Now Gson knows to produce List<Order>, not List<Any>
val orders: List<Order> = gson.fromJson(json, type)

// The same pattern for any generic type:
val mapType = object : TypeToken<Map<String, Order>>() {}.type
val setType = object : TypeToken<Set<Int>>() {}.type
```

The 'object : TypeToken<List<Order>>() {}' syntax creates an anonymous subclass of TypeToken. Because it is a subclass (not TypeToken itself), Kotlin preserves the generic argument <List<Order>> in the class definition, making it available at runtime.

Flutter Equivalent

In Dart, type erasure does not exist in the same way -- Dart's type system is reified (types are preserved at runtime). So deserializing a list is straightforward:

Dart / Flutter

```
// Dart -- no TypeToken needed, cast directly
final jsonList = jsonDecode(json) as List<dynamic>;
final orders = jsonList.map((e) => Order.fromJson(e)).toList();

// This works because Dart preserves List<dynamic> at runtime
// Java/Kotlin cannot do this -- List<Order> becomes List at runtime
```

Quick Reference: When to Use TypeToken

What you are deserializing	Code
Single object	gson.fromJson(json, Order::class.java)
List of objects	val t = object : TypeToken<List<Order>>() {}.type
Map	val t = object : TypeToken<Map<String,Order>>() {}.type
Nested generic	val t = object : TypeToken<List<Map<String,Order>>>() {}.type

Chapter 4: Storing Order History -- The Full Pattern

The Read-Modify-Write Pattern

SharedPreferences stores one value per key. To maintain a list (like order history), you use the read-modify-write pattern on every write:

```
1. Read  -- get the current JSON string from SharedPreferences
2. Decode -- convert JSON string to List<Order> using Gson
3. Modify -- add/remove/update the item in the list
4. Encode -- convert the updated list back to JSON string using Gson
5. Write  -- save the new JSON string back to SharedPreferences
```

Full Implementation in This Project

Kotlin / Jetpack Compose

```
class OrderRepositoryImpl @Inject constructor(
    private val sharedPreferencesManager: SharedPreferencesManager,
) : IOrderRepository {

    companion object {
        private const val KEY_ORDER_HISTORY = "order_history"  // the storage key
    }

    private val gson = Gson()

    override suspend fun placeOrder(items: List<CartItem>): Order {
        delay(4000)  // simulate network
        val order = Order(id = orderIdCounter++, items = items, ...)
        saveOrderToHistory(order)  // persist to SharedPreferences
        return order
    }

    override fun getOrderHistory(): List<Order> {
        // STEP 1: Read -- get stored JSON (empty string if nothing saved yet)
        val json = sharedPreferencesManager.getString(KEY_ORDER_HISTORY)
        if (json.isEmpty()) return emptyList()

        // STEP 2: Decode -- TypeToken needed for List<Order>
        val type = object : TypeToken<List<Order>>() {}.type
        return gson.fromJson(json, type)
    }

    private fun saveOrderToHistory(order: Order) {
        // STEP 1: Read current list
```

```
        val currentHistory = getOrderHistory().toMutableList()

        // STEP 2: Modify -- add new order at position 0 (newest first)
        currentHistory.add(0, order)

        // STEP 3: Encode + Write
        val json = gson.toJson(currentHistory)
        sharedPreferencesManager.putString(KEY_ORDER_HISTORY, json)
    }
}
```

Flutter Equivalent -- Full Pattern

Dart / Flutter

```
class OrderRepositoryImpl implements IOrderRepository {
  static const _keyOrderHistory = 'order_history';

  @override
  Future<Order> placeOrder(List<CartItem> items) async {
    await Future.delayed(Duration(seconds: 4));  // simulate network
    final order = Order(id: _counter++, items: items, ...);
    await _saveOrderToHistory(order);
    return order;
  }

  Future<List<Order>> getOrderHistory() async {
    final prefs = await SharedPreferences.getInstance();

    // STEP 1: Read
    final json = prefs.getString(_keyOrderHistory);
    if (json == null) return [];

    // STEP 2: Decode -- Dart requires manual fromJson on each item
    final list = jsonDecode(json) as List<dynamic>;
    return list.map((e) => Order.fromJson(e)).toList();
  }

  Future<void> _saveOrderToHistory(Order order) async {
    final prefs = await SharedPreferences.getInstance();

    // STEP 1: Read
    final current = await getOrderHistory();

    // STEP 2: Modify
    final updated = [order, ...current];  // newest first

    // STEP 3: Encode + Write
```

```
    await prefs.setString(_keyOrderHistory, jsonEncode(
        updated.map((e) => e.toJson()).toList()
    ));
  }
}
```

Key Differences Between Dart and Kotlin

Point	Dart	Kotlin + Gson
Get prefs instance	async -- await SharedPreferences.getInstance()	sync -- injected via Hilt @Singleton
Object -> JSON	manual toJson() required	gson.toJson(obj) automatic
JSON -> Object	manual fromJson() required	gson.fromJson(json, Class) automatic
JSON -> List	cast + map loop	TypeToken + gson.fromJson
Type erasure	Not an issue (Dart is reified)	Requires TypeToken for generics
Write result	async -- returns Future<bool>	apply() is fire-and-forget

Chapter 5: SharedPreferences vs DataStore

Why DataStore Exists

SharedPreferences was Android's original storage API, but it has serious problems in modern apps. Jetpack introduced DataStore as its replacement.

Feature	SharedPreferences	DataStore Preferences
Thread safety	No -- can cause issues on main thread	Yes -- fully coroutine-based
Async API	No -- synchronous (blocking)	Yes -- suspend / Flow
Reactive updates	No -- must poll for changes	Yes -- Flow emits on every change
Type-safe keys	No -- raw string keys	Yes -- typed key objects
Error handling	Silent failures	Exceptions propagate via Flow
Jetpack recommended	No (legacy)	Yes (current standard)

DataStore Example -- Same Pattern as SharedPreferences

Kotlin / Jetpack Compose

```
// Define typed keys -- no string typos possible
val USERNAME_KEY = stringPreferencesKey("username")
val ORDER_COUNT_KEY = intPreferencesKey("order_count")

// Write -- suspend fun, runs off main thread automatically
suspend fun saveUsername(dataStore: DataStore<Preferences>, name: String) {
    dataStore.edit { prefs ->
        prefs[USERNAME_KEY] = name
    }
}

// Read -- returns a Flow, reactive
val username: Flow<String> = dataStore.data
    .map { prefs -> prefs[USERNAME_KEY] ?: "" }

// Collect in ViewModel as StateFlow
val username: StateFlow<String> = dataStore.data
    .map { prefs -> prefs[USERNAME_KEY] ?: "" }
    .stateIn(viewModelScope, SharingStarted.WhileSubscribed(), "")
```

When to Use Each

Use case	Recommended
Simple flags, user preferences, settings	DataStore (modern)
Quick prototyping, learning projects	SharedPreferences (simpler API)
Reactive UI that updates when prefs change	DataStore (Flow support)

Legacy code maintenance	SharedPreferences (already there)
New production app	DataStore (Jetpack standard)

[!] For the order history feature in this project, SharedPreferences is used for simplicity and learning. In a production app, order history would be stored in a Room database (SQLite) instead -- better querying, relationships, and migrations.

Chapter 6: Quick Reference Cheat Sheet

SharedPreferences Cheat Sheet

```
// Get instance (via wrapper or directly)
val prefs = context.getSharedPreferences("name", Context.MODE_PRIVATE)

// Write
prefs.edit().putString("key", "value").apply()    // String
prefs.edit().putInt("key", 42).apply()             // Int
prefs.edit().putBoolean("key", true).apply()       // Boolean
prefs.edit().putFloat("key", 3.14f).apply()        // Float
prefs.edit().putLong("key", 123L).apply()          // Long

// Read
val str  = prefs.getString("key", "default")
val num  = prefs.getInt("key", 0)
val flag = prefs.getBoolean("key", false)

// Delete
prefs.edit().remove("key").apply()    // one key
prefs.edit().clear().apply()           // all keys

// Check
val exists = prefs.contains("key")
```

Gson Cheat Sheet

```
val gson = Gson()

// Single object -> JSON
val json: String = gson.toJson(myObject)

// JSON -> Single object
val obj: MyClass = gson.fromJson(json, MyClass::class.java)

// List -> JSON
val json: String = gson.toJson(myList)

// JSON -> List (TypeToken required!)
val type = object : TypeToken<List<MyClass>>() {}.type
val list: List<MyClass> = gson.fromJson(json, type)

// JSON -> Map
val mapType = object : TypeToken<Map<String, MyClass>>() {}.type
```

```
val map: Map<String, MyClass> = gson.fromJson(json, mapType)
```

Order History Pattern Cheat Sheet

```
companion object { private const val KEY = "order_history" }
private val gson = Gson()

// Save (read-modify-write)
fun save(order: Order) {
    val list = load().toMutableList()
    list.add(0, order)                                  // newest first
    prefs.putString(KEY, gson.toJson(list))            // write back
}

// Load
fun load(): List<Order> {
    val json = prefs.getString(KEY)
    if (json.isEmpty()) return emptyList()
    val type = object : TypeToken<List<Order>>() {}.type
    return gson.fromJson(json, type)
}

// Delete one
fun delete(orderId: Int) {
    val list = load().toMutableList()
    list.removeAll { it.id == orderId }
    prefs.putString(KEY, gson.toJson(list))
}

// Clear all
fun clearHistory() = prefs.remove(KEY)
```

Dart vs Kotlin Full Comparison

Task	Dart	Kotlin
Persist key-value	shared_preferences package	SharedPreferences (built-in)
Object to JSON	jsonEncode(obj.toJson())	gson.toJson(obj)
JSON to object	MyClass.fromJson(jsonDecode(s))	gson.fromJson(s, MyClass::class.java)
JSON to List	(jsonDecode(s) as List).map(fromJson)	TypeToken + gson.fromJson
toJson/fromJson manual?	Yes (or use json_serializable)	No (Gson reflects automatically)
Modern replacement	hive / isar / ObjectBox	DataStore / Room